Cálculo Lambda

Marco A L Barbosa
malbarbo.pro.br

Departamento de Informática
Universidade Estadual de Maringá

Este trabalho está licenciado com uma Licença Creative Commons - Atribuição-CompartilhaIgual 4.0 Internacional.
http://github.com/malbarbo/na-progfun

O cálculo lambda

O cálculo lambda

- O cálculo lambda é um modelo de computação introduzido por Alonzo Church na década de 1930
  - Qualquer função computável pode ser expressa e avaliada utilizando essa formulação
  - Possui poder equivalente às máquinas de Turing
- Consiste de uma única regra de transformação (substituição de variáveis) e um único esquema de definição de funções

Termos lambda

- O cálculo lambda possui três componentes, ou termos lambda, básicos:
  - Abstrações
  - Variáveis
  - Expressões

Termos lambda

- Variáveis são apenas nomes, representadas por uma letra minúscula
  - $a, b, c, \ldots, x, y, z$
- Uma "expressão" é um superconjunto dos termos lambda
  - Pode ser uma variável, uma abstração, ou uma combinação de ambas.
- Gramática da linguagem: $e = x|(\lambda x.e)|e(e)$

Abstração (função)

- A abstração é uma **função**
- É composta por duas partes: *cabeça* e *corpo*
  - A cabeça consiste de um λ seguido de uma variável
  - O corpo é uma expressão
  - Um ponto "." separa a cabeça do corpo
- Exemplo: $\lambda x.x$
  - Essa abstração representa a função identidade
    - Na matemática: $f(x) = x$
  - A variável após o λ é o identificador do argumento da função

Abstração (função)

- As funções são anônimas (não possuem nome)
- O nome do argumento na definição de função é apenas um “espaço reservado”, podendo ser renomeado

$$(\lambda z.z) \equiv (\lambda y.y) \equiv (\lambda t.t) \equiv (\lambda u.u)$$

- Isso é chamado de “equivalência-α”

Redução-β (aplicação)

- Quando aplicamos uma função a um argumento, substituímos todas as instâncias da variável ligada no corpo pelo argumento
- A substituição elimina a cabeça da abstração, restando apenas o corpo substituído
  - Esse processo é chamado de "redução-β"
- Exemplo: aplicação da função identidade sobre o argumento 2:

$$(\lambda x.x)2 \rightarrow x[x := 2] = 2$$

  - A aplicação $(\lambda x.x)2$ reduz para $x[x := 2]$ que resulta em 2

Redução-β (aplicação)

Outros exemplos:

- $(\lambda x.x + 1)2 \to x + 1[x := 2] = 2 + 1 \Rightarrow 3$
- $(\lambda x.x)(\lambda y.y) \to x[x := (\lambda y.y)] = \lambda y.y \equiv \lambda x.x$
- $((\lambda x.x)(\lambda y.y))z \to x[x := (\lambda y.y)] = (\lambda y.y)z \to y[y := z] = z$

Expressões

- Uma expressão pode ser cercada com parênteses para melhorar a legibilidade
  - Se E é uma expressão, então (E) é a mesma expressão
- Para não sobrecarregar expressões com parênteses, adotamos que:
  - A aplicação é associativa à esquerda:

$$MNO \equiv (MN)O$$

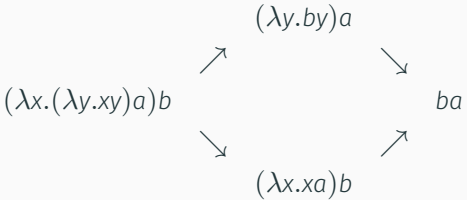
  - O corpo de uma abstração se estende ao máximo à direita:

$$\lambda x.MN \equiv \lambda x.(MN) \text{ e não } (\lambda x.M)N$$

- Podemos utilizar os parênteses para alterar a ordem das aplicações

- A avaliação de uma expressão consiste na sua redução sucessiva até que ela atinja uma **forma normal**
  - Uma expressão que não pode mais ser reduziada é dita estar na forma normal
- Nem toda expressão possui uma forma normal:
  - $(\lambda x.xx)(\lambda y.yy)$

- A avaliação de uma expressão pode seguir caminhos distintos de redução, mas produzindo o mesmo termo final



Variáveis livres e ligadas

- As variáveis livres de um termo são as variáveis que não estão ligadas por uma abstração lambda
  - A variável x é livre em x
  - As variáveis livres de $\lambda x.M$ são as variáveis livres de M, com x removido
  - As variáveis livres de MN são a união das variáveis livres de M e N

Variáveis livres e ligadas

- Exemplos
  - Em $\lambda x.xy$, x está ligada e y é livre
  - Em $\lambda x.(\lambda y.xy)$, x e y estão ligadas
  - Em $(\lambda x.xt)(\lambda y.yz)$, t e z são livres

Variáveis livres e ligadas

- No cálculo lambda todos os nomes são locais às definições
- Em $(\lambda x.x)(\lambda y.yx)$, x está ligada na primeira abstração, mas livre na segunda
- Os x da primeira e segunda expressões são totalmente independentes

- Devemos tomar cuidado durante as substituições para não misturar as ocorrências livres e ligadas de uma variável
- Na redução da expressão $(\lambda x.(\lambda y.xy))y$
  - Uma substituição direta poderia misturar os dois identificadores y, resultando em $\lambda y.yy$
  - Renomeando o y vinculado por t, obtemos a avaliação correta:

$$(\lambda x.(\lambda t.xt))y \to (\lambda t.xt)[x := y] = \lambda t.yt \equiv \lambda x.yx$$

Currying

- Por definição, a abstração lambda permite apenas uma variável de entrada
- Podemos ampliar o número de entradas por meio de currying
  - A ideia é construir funções “aninhadas” que consumam uma entrada de cada vez
  - Cada função deve resultar em uma nova função com a entrada consumida substituída em seu corpo

- Considere uma função que deve ser aplicada sobre dois números e resultar na soma entre eles

$$\lambda x.(\lambda y.x + y)$$

- Ao aplicarmos a função sobre dois argumentos sucessivamente (digamos 2 e 3), obtemos:

$$\begin{aligned}(\lambda x.(\lambda y.x + y)2)3 &\to ((\lambda y.x + y)[x := 2])3 = (\lambda y.2 + y)3 \\ &\to (2 + y)[y := 3] = 2 + 3 \Rightarrow 5\end{aligned}$$

- Funções que possuem o padrão $\lambda x.\lambda y.\lambda z.\cdots.E$ são caracterizadas pelo currying
  - Por conveniência, elas podem ser abreviadas por $\lambda xyz\cdots.E$

Aritmética

Aritmética

- No cálculo-λ só podemos criar novas funções
  - Não há pré-definições de dados nem operações (embora tenhamos usado números e operações aritméticas em nossos exemplos)

- Devemos definir os números e operações como funções
- Podemos inicialmente definir o número 0 como: $\lambda s.(\lambda z.z)$
  - Ou de maneira abreviada: $\lambda sz.z$
- Os demais números naturais são sucessões do 0:

  $1 \equiv \lambda sz.sz$

  $2 \equiv \lambda sz.s(sz)$

  $3 \equiv \lambda sz.s(s(sz))$

  E assim por diante ...

- A função sucessora pode ser definida como:

$$\mathsf{S} \equiv \lambda wyx.y(wyx)$$

- Aplicando S à representação do 0 temos:

$$\mathsf{S0} \equiv (\lambda wyx.y(wyx))(\lambda sz.z)$$

- Avaliando a expressão S0, temos:

$$\lambda yx.y((\lambda sz.z)yx) = \lambda yx.y((\lambda z.z)x) = \lambda yx.y(x) \equiv \lambda sz.sz \equiv 1$$

- Como exercício, verifique que a aplicação S(S0) avalia para 2

Adição

- A adição pode ser obtida diretamente ao verficarmos que no corpo de 1, sz representa a aplicação da função s sobre z
- Se quisermos representar a soma $2 + 3$ por exemplo, basta aplicarmos 2 vezes a função sucessora sobre 3
- Assim, para computar $2 + 3$, temos:

$$2S3 \equiv (\lambda sz.s(sz))(\lambda wyx.y(wyx))(\lambda uv.u(u(uv)))$$

- A qual reduz para:

$$(\lambda wyx.y((wy)x))((\lambda wyx.y((wy)x))(\lambda uv.u(u(uv)))) \equiv S(S3)$$

- Como exercício, verifique que S(S3) reduz para S4 = 5

Multiplicação

- A multiplicação entre dois números pode ser computada utilizando a seguinte função:

$$\mathsf{M} \equiv \lambda xyz.x(yz)$$

- Assim, o produto de 2 por 2 é M22
- Reduzindo a expressão, obtemos $\lambda z.2(2z)$
  - Como exercício, verifique que essa expressão reduz para 4

Outras funções

- Faltam muitas outras funções para representar as possíveis computações que conhecemos:
  - Operações lógicas
  - Operações relacionais
  - Condicionais
  - Recursão
  - Estruturas de dados como pares, listas, etc.
  - Dentre outras ...
- Todas elas podem ser expressas em cálculo lambda
- Consulte o material TILC para saber como elas podem ser representadas

Leitura recomendada

Leitura recomendada

- λ-Calculus: The Other Turing Machine
- A Tutorial Introduction to the Lambda Calculus